

L4 • TOKEN-AS-A-SERVICE •

Pay-per-token API

The unit we hand people changed. Not a server – a token, with a price on it.

L3 • INFERENCE SERVING •

Model routing & scaling

This is where a GPU-hour becomes a token.
The **engines** come from the PyTorch Foundation;
the **serving control plane** comes from CNCF.

L2 • ORCHESTRATION •

GPU pooling & scheduling

Twenty-five hundred GPUs, and most of them idle most of the time. This layer is where that stopped.

L1 • BARE METAL •

Physical GPU servers

One fleet. Every class of AI workload.
One way to provision all of it.

From **BARE METAL** to **Token-as-a-Service**

A four-layer cloud-native stack that turned a fragmented enterprise GPU fleet into a metered token service - assembled almost entirely from projects in this room.



Enterprise GPU Fleet



Metered Token Service



4-Layer Cloud-Native Stack



Trong Vinh. Nguyen (Tovin)
阮仲永

The limit was never the hardware. It was the **unit of allocation.**

Like most enterprises, we allocated AI infrastructure the way we allocate servers: one team, one machine.

Dedicated hardware silos

One team, one set of machines. The standard model everywhere — and it makes capacity impossible to share.

And inside each silo, capacity goes unused

A service is given hardware sized for its peak, then spends most of its life far below it — and no one else can use the difference.

More hardware does not change the shape

New workloads still wait on procurement, because the constraint is how capacity is divided — not how much of it there is.

WHAT THAT COSTS IN PRACTICE

~10%

COMPUTE UTILISATION — ONE FLAGSHIP GPU RUNNING
ONE INFERENCE SERVICE AT FULL LOAD

Small inference services sit far below what a modern accelerator can do, and we could not tune our way past it. The headroom is real. It just is not reachable one workload at a time.

L1 Bare Metal

One fleet. Every class of AI workload. **One way to provision all of it.**

~2,500

GPUs under one platform, across many hardware generations

NCP 1K

GPU cluster, built to NVIDIA's Cloud Partner reference architecture

SuperPOD + BasePOD

NVIDIA reference architectures running in production

WHAT RUNS ON IT

Frontier training

multi-node, SuperPOD-scale, InfiniBand

LLM & agentic inference

the top-end cards, kept busy

Small inference • notebooks • batch

thousands that never fill a card

WHAT THIS LAYER DELIVERS

- Bare Metal GPU
- VM GPU
- GPU Container
- Kubernetes Cluster
- AI Super Cluster
- AI Flash Storage

HOW IT IS BUILT

- OpenStack Ironic
- OpenStack Nova
- Kata Containers
- GPU Operator
- Network Operator

A mixed fleet – current-generation silicon down to cards several years old – all provisioned one identical way.

L1 • Bare Metal	L2 • Orchestration	L3 • Inference Serving	L4 • Token-as-a-Service
-----------------	--------------------	------------------------	-------------------------

L2 Orchestration

Twenty-five hundred GPUs, and most of them idle most of the time. This layer is where that stopped.

We didn't buy capacity. We recovered it.

Same racks, same power budget, same procurement. The only thing that changed was how much of each card actually gets used.

THE CAUSE

13% → 59%

GPU utilisation on one card, same workload, same SLA

EVERYTHING ELSE FOLLOWS FROM IT

3.5x = 71%

more work per card is fewer cards for the same throughput – one result, stated two ways

EFFICIENCY GAINS

2.1x

work delivered per Watt – power rose far less than output

WHAT MAKES IT WORK

A GPU is no longer the unit we allocate – **a slice of one is**. Each workload gets a memory and compute quota, many workloads pack onto a card, and usage is attributed **per workload, not per device** – which is what makes chargeback and right-sizing possible at all.

TECHNOLOGY PLATFORM

- **Kubernetes**
 - **HAMi** GPU Slicing
 - **KEDA** Autoscaling
 - **Cilium**
 - Prometheus
 - Argo CD
- Harbor
 - Kubeflow
 - KAI Scheduler
 - Grafana

Measured on a single production GPU – same card, same workload, same service level.

L1 • Bare Metal

L2 • Orchestration

L3 • Inference Serving

L4 • Token-as-a-Service

L3 Inference Serving

This is where a GPU-hour becomes a token. The **engines** come from the PyTorch Foundation; the **serving control plane** comes from CNCF.

ENGINES — PYTORCH FOUNDATION

- vLLM
- PyTorch
- SGLang PyTorch ecosystem

SERVING CONTROL PLANE — CNCF

- KServe
- ILM-d
- Knative
- NVIDIA Dynamo on NATS + etcd

THE ENGINEERING JUDGEMENT BEHIND THIS LAYER

Share a GPU when one workload **cannot fill it.
Give it the whole card when one workload **already** can.**

The test is one question, not a benchmark: push load and watch whether utilisation climbs to the hardware ceiling. If it does, the card is already full — sharing only divides it. If throughput flattens while the card stays idle, sharing is free capacity. The figures below are ours, on our workloads — the rule is what transfers.

A service that cannot batch

request size capped upstream — one instance never moved past ~10% of the card

Share it. 3.3x on our forecasting workload

A model that batches continuously

the engine already keeps the card busy on its own

Don't share it. splitting cost us 1.6x

Compute-bound training

packing divides the same silicon rather than filling idle silicon

Share only for isolation. 0.77x aggregate

L1 • Bare Metal

L2 • Orchestration

L3 • Inference Serving

L4 • Token-as-a-Service

L4 Token-as-a-Service

The unit we hand people changed. Not a server — **a token, with a price on it.**

BEFORE

File a ticket. Wait weeks. Receive a physical server. Hold it forever.

AFTER

Self-service an API key. Get an OpenAI-compatible endpoint in minutes. Pay for what you consume.



GOVERNANCE IS BUILT INTO THE GATEWAY, NOT BOLTED ON

Authentication, quota and token metering all run **inside the gateway itself** — not as services bolted on beside it. **A shared control plane fronts one or more regional data planes**, so inference traffic, authorisation and usage records all stay inside the region that served them. Sovereign by design — models, data and compute stay in-country.

TECHNOLOGY PLATFORM

- Envoy AI Gateway
- Envoy Gateway
- Gateway API Inference Extension
- Strimzi · Kafka

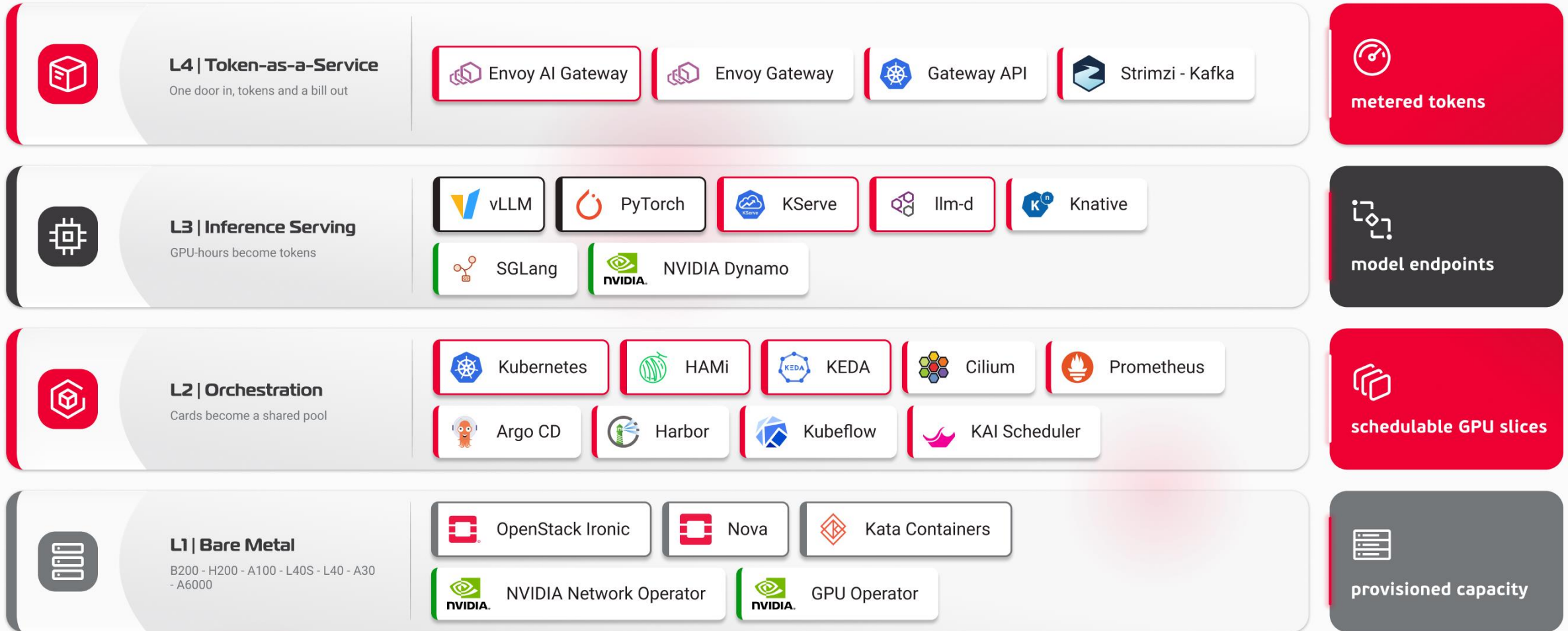
L1 · Bare Metal	L2 · Orchestration	L3 · Inference Serving	L4 · Token-as-a-Service
-----------------	--------------------	------------------------	-------------------------

We did not build a stack

We wired one **together**

THREE FOUNDATIONS • FOUR LAYERS • ALL IN PRODUCTION

Viettel Platform



One fleet, one pool, a factory that ships tokens.

71%

fewer GPUs, same throughput
– on the workload we measured

minutes

for a team to get AI capacity
– it used to be weeks

1

front door – one API for
every team, model and region

~2,500

GPUs pooled
under one platform

- **The unit of allocation changed, and everything followed.** Server → Slice → Token. Each step turned existing capacity into more served work.
- **We bought almost none of this.** Kubernetes, HAMi, KEDA, Envoy AI Gateway, KServe, llm-d, vLLM, OpenStack.
We wrote the glue and file what we find back upstream.
- **Next: the same pool serving agents, not just requests.** Multi-agent workloads are bursty and numerous — exactly the shape this architecture was built for.

We didn't buy this stack. We joined it.

Every layer you saw runs on projects in this room — and what we learn running them goes back to them.



Thank you.

Let's build **the future of AI.**

From bare metal to token — every layer you saw runs in production today.

4

PLATFORM LAYERS

~2,500

GPUS POOLED

1

UNIFIED API

<5min

FROM CODE TO SERVING

The stack is live. The APIs are open.

Escaping the Vendor Trap

A Journey for Migrating Legacy Infrastructure to OpenStack and K8s

Speaking at KubeCon + CloudNativeCon + OpenInfra Summit + PyTorch Conference China 2026



Trong Vinh. Nguyen (Tovin)

Deputy Director at Platforms and Digital Transformation Center



Quoc Dat. Le

Cloud Principal Specialist

Please keep in mind
Sep 9th 2026
11:00 - 11:30
Grand Ballroom II + III